MIP_GOOGLE_ADK_IMPLEMENTATION_GUIDE.md

Mainframe Intelligence Platform (MIP)

Google ADK Implementation Guide

Version: 1.0

Status: Implementation Blueprint

Classification: Engineering Guide

Owner: MIP Engineering Team

Executive Summary

This document defines how MIP will be implemented using:

- Google ADK
- Python
- SQLite
- NetworkX
- Pydantic
- GitHub Copilot
- VS Code

This guide serves as the engineering blueprint for MIP v1.0.

Objectives

Build an enterprise-grade modernization intelligence platform capable of:

- Repository Discovery
- Mainframe Code Analysis
- Metadata Extraction
- Knowledge Graph Construction
- Lineage Analysis
- Impact Analysis
- Reasoning
- Modernization Recommendations

Technology Stack

Core Stack

```
Python 3.12+

Google ADK

SQLite

NetworkX

Pydantic

Typer

Pytest
```

Future Stack

```
PostgreSQL

Neo4j

Kafka

ElasticSearch

Vector Database

Kubernetes
```

Repository Structure

```
mip-platform/

├── .github/
│   └── copilot-instructions.md
│
├── docs/
│   ├── architecture/
│   ├── implementation/
```

```
│   ├── leadership/
│   └── roadmap/
│
├── prompts/
│   ├── discovery/
│   ├── parsing/
│   ├── metadata/
│   ├── graph/
│   ├── lineage/
│   ├── impact/
│   ├── reasoning/
│   └── modernization/
│
├── skills/
│
├── src/
│   └── mip/
│
│       ├── agents/
│       ├── discovery/
│       ├── parsing/
│       ├── metadata/
│       ├── graph/
│       ├── lineage/
│       ├── impact/
│       ├── reasoning/
│       ├── modernization/
│       ├── repositories/
│       ├── models/
│       ├── services/
│       ├── workflows/
│       └── cli/
│
├── tests/
│
├── data/
│
├── sqlite/
│
└── mfcode/
```

Multi-Agent Architecture

```
Repository

↓
```

```
Discovery Agent

↓

Parser Agents

↓

Metadata Agent

↓

Graph Agent

↓

Lineage Agent

↓

Impact Analysis Agent

↓

Reasoning Agent

↓

Modernization Architect Agent
```

Agent Registry

```
AGENT_REGISTRY = {

    "discovery": DiscoveryAgent,

    "cobol_parser": CobolParserAgent,

    "jcl_parser": JCLParserAgent,

    "copybook_parser": CopybookParserAgent,

    "sql_parser": SQLParserAgent,

    "metadata": MetadataAgent,
```

```
    "graph": GraphBuilderAgent,

    "lineage": LineageAgent,

    "impact": ImpactAnalysisAgent,

    "reasoning": ReasoningAgent,

    "modernization": ModernizationArchitectAgent
}
```

Base Agent Contract

```
from abc import ABC
from abc import abstractmethod

class BaseAgent(ABC):

    @abstractmethod
    def run(self, context):
        pass

    @abstractmethod
    def validate(self):
        pass

    @abstractmethod
    def explain(self):
        pass
```

Shared Context Model

```
from pydantic import BaseModel

class MIPContext(BaseModel):

    repository_path: str

    inventory: dict = {}

    metadata: dict = {}

    graph: object = None
```

```
    lineage: dict = {}

    impact: dict = {}

    recommendations: list = []
```

Discovery Layer

DiscoveryAgent

Purpose:

- Repository Scanning
- Technology Detection
- Asset Inventory

Outputs:

```
{
  "programs": [],
  "copybooks": [],
  "jcls": [],
  "sql_files": []
}
```

RepositoryScannerAgent

Responsibilities:

- Recursive Scan
- File Identification
- Folder Classification

FileClassifierAgent

Responsibilities:

- COBOL Detection
- JCL Detection
- Copybook Detection
- SQL Detection
- Java Detection

Parsing Layer

CobolParserAgent

Extract:

- Program Metadata
- CALL Statements
- File Usage
- SQL Usage
- Business Rules

Output:

```
{
  "program_name": "",
  "calls": [],
  "reads": [],
  "writes": []
}
```

JCLParserAgent

Extract:

- Jobs
- Steps
- Program Executions
- Datasets

CopybookParserAgent

Extract:

- Fields
- Layouts
- Data Types

SQLParserAgent

Extract:

- Tables

- Columns
- CRUD Operations

Metadata Layer

MetadataAgent

Responsibilities:

- Normalize Metadata
- Deduplicate Records
- Validate Relationships

Output:

```
Canonical Metadata Model
```

Storage:

```
SQLite
```

Graph Layer

RelationshipAgent

Generate:

```
CALLS

READS

WRITES

USES

EXECUTES
```

GraphBuilderAgent

Technology:

```
import networkx as nx

graph = nx.MultiDiGraph()
```

Outputs:

```
Knowledge Graph

Graph Metrics

Communities
```

Lineage Layer

LineageAgent

Generate:

```
Data Lineage

Execution Lineage

Business Lineage
```

Algorithms:

```
nx.shortest_path()

nx.descendants()

nx.ancestors()

nx.all_simple_paths()
```

Impact Analysis Layer

ImpactAnalysisAgent

Responsibilities:

- Blast Radius Analysis

- Dependency Analysis
- Criticality Assessment

Output:

```
{
  "impact_score": 89,
  "severity": "CRITICAL"
}
```

Reasoning Layer

ReasoningAgent

Purpose:

Convert metadata and graph relationships into actionable intelligence.

Questions:

- What is important?
- What is risky?
- What should be modernized first?

Outputs:

```
Recommendations

Capability Assignments

Risk Assessments

Migration Suggestions
```

Modernization Layer

CapabilityDiscoveryAgent

Discover:

- Domains
- Capabilities
- Ownership

ServiceDiscoveryAgent

Discover:

- Microservice Candidates
- API Candidates
- Event Candidates

MigrationPlanningAgent

Generate:

```
Wave 1

Wave 2

Wave 3

Wave 4
```

ModernizationArchitectAgent

Produce:

- Target Architecture
- Migration Roadmap
- Modernization Strategy

SQLite Integration

Database:

```
mip.db
```

Core Tables:

```
program

copybook
```

```
job

dataset

table_metadata

relationship

lineage

impact

recommendation
```

NetworkX Integration

Use:

```
graph = nx.MultiDiGraph()
```

Reason:

A node may have multiple relationships.

Example:

```
TS2AUTH

READS CARD_MASTER

WRITES CARD_MASTER

CALLS TS2VALID

USES CPY_CARD
```

Workflow Orchestration

Repository Analysis

```
Repository
```

```
↓

Discovery

↓

Parsing

↓

Metadata

↓

Graph

↓

Lineage

↓

Impact

↓

Reasoning

↓

Modernization
```

Google ADK Workflows

Recommended workflows:

```
repository_analysis

metadata_generation

graph_construction

lineage_analysis

impact_analysis
```

```
modernization_recommendation
```

Logging Strategy

Every agent logs:

```
{
  "agent": "",
  "status": "",
  "duration": "",
  "confidence": ""
}
```

Explainability Requirements

Every result must include:

```
{
  "result": {},
  "confidence": 0.95,
  "evidence": [],
  "reasoning": []
}
```

No black-box decisions.

Phase 1 Deliverables

- Repository Discovery
- COBOL Parsing
- JCL Parsing
- Copybook Parsing
- Metadata Generation
- SQLite Storage
- NetworkX Graph Construction
- Lineage Generation
- Impact Analysis
- HTML Report Generation

Phase 2 Deliverables

- Google ADK Integration
- Multi-Agent Orchestration
- Capability Discovery
- Service Discovery
- Modernization Recommendations

Phase 3 Deliverables

- LLM-Assisted Reasoning
- Business Rule Discovery
- Migration Wave Planning
- Executive Dashboards

Success Criteria

A user executes:

```
mip analyze ./mfcode
```

Expected Output:

```
Programs: 12,842

Jobs: 1,334

Copybooks: 4,221

Relationships: 186,420

Capabilities: 43

Service Candidates: 28

Critical Components: 112

Modernization Roadmap Generated
```

MIP Principle

Agents discover.

Metadata organizes.

Graphs connect.

Lineage explains.

Impact protects.

Reasoning guides.

Modernization transforms.

End of Document